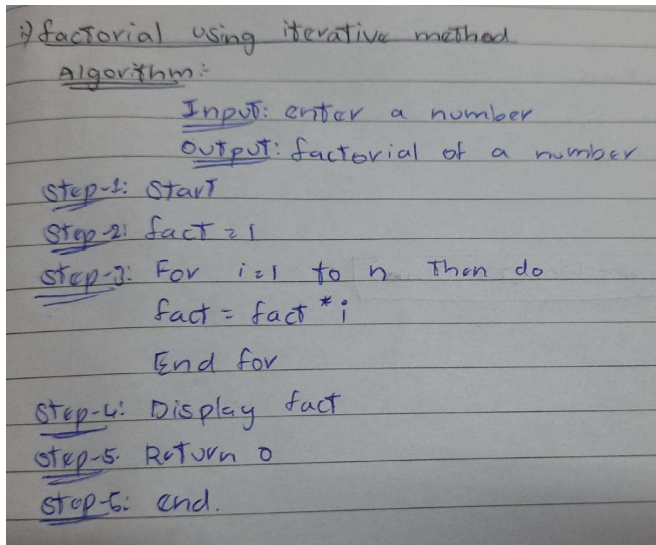


PRACTICAL-04

AIM: Implementation and Time analysis of factorial program using iterative and recursive method

1.) ITERATIVE METHOD

ALGORITHM:



Factorial using iterative method
Algorithm:
Input: enter a number
Output: factorial of a number
Step-1: Start
Step-2: fact = 1
Step-3: For i = 1 to n. Then do
 fact = fact * i
 End for
Step-4: Display fact
Step-5: Return 0
Step-6: end.

PROGRAM:

```
#include <iostream>

using namespace std;

int main(){
    int n;

    int fact = 1;

    cout << "enter a number:";

    cin >> n;

    for(int i = 1; i <= n; i++){
        fact = fact * i;
    }

    cout << "Factorial of " << n << " = " << fact;

    return 0;
}
```

OUTPUT:

```
Output
enter a number:5
Factorial of 5 = 120

=== Code Execution Successful ===
```

TIME COMPLEXITY:

Time Complexity

~~Time~~

fact = 1 $\rightarrow 1$
for i = 1 to n $\rightarrow n$
fact = fact * i $\rightarrow n$
 $T(n) = 1 + n + n$
 $= 2n + 1$
 $T(n) = O(n)$

i) Best case
loop runs from 1 to n
fact = 1 $\rightarrow 1$
for i = 1 to n $\rightarrow n$
fact = fact * i $\rightarrow n$
 $T(n) = 1 + n + n = 2n + 1$
 $T(n) = O(n)$

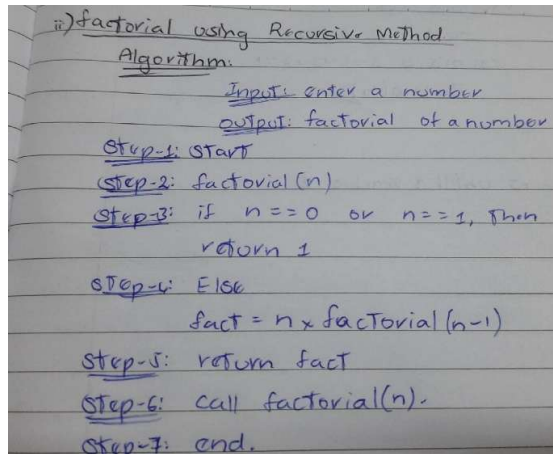
ii) Average case
loop always executes n times
i = 1 $\rightarrow 1$
i = 2 $\rightarrow 2$
i = 3 $\rightarrow 3$
i = n $\rightarrow 1$ no. of iteration = n
So, $T(n) = O(n)$

iii) Worst case
It is also, loop executes n times
 $n \rightarrow n/2, \rightarrow n/4, \dots$
1/2, 3/4, ... n
 $T(n) = 1 + n + n = 2n + 1$
 $T(n) = O(n)$

Best = Average = Worst = $O(n)$

2.) RECURSIVE METHOD:

ALGORITHM:



ii) factorial using Recursive Method
Algorithm:
Input: enter a number
Output: factorial of a number
Step-1: Start
Step-2: factorial(n)
Step-3: if $n == 0$ or $n == 1$, then
return 1
Step-4: Else
fact = $n \times \text{factorial}(n-1)$
Step-5: return fact
Step-6: call factorial(n).
Step-7: end.

PROGRAM:

```
#include <iostream>

using namespace std;

class factorial{
public:
    int factr(int n){
        if(n == 0 || n == 1){
            return 1;
        }
        else{
            int result = n*factr(n-1);
            return result;
        }
    }
};

int main(){
    int n,result;
    factorial obj;

    cout << "enter a number:";

    cin>>n;

    result = obj.factr(n);

    cout << "factorial of " << n << " is " << result;
```

```
return 0;  
}
```

OUTPUT:

```
Output  
enter a number:7  
factorial of 7 is 5040  
  
=== Code Execution Successful ===
```

TIME COMPLEXITY:

Time Complexity
factorial(n) $\rightarrow n$
if (n == 0 || n == 1) $\rightarrow n$
n * factorial(n-1) $\rightarrow (n-1)$
return fact $\rightarrow n$

 $T(n) = n + n + (n-1) + n$
 $= 4n - 1$
 $T(n) = O(n)$
It is worst case Time complexity = $O(n)$

i) Best case:
when $n = 0$ or $n = 1$
factorial(1) $\rightarrow 1$
 $T(n) = O(1)$

ii) Average case:
no. of calls is 5
ex: $n = 5 \rightarrow 5 \rightarrow 4 \rightarrow 3 \rightarrow 2 \rightarrow 1$
 $T(n) = O(n)$

iii) Worst case:
large number of n continues until it reaches 1
ex: $n \rightarrow n-1 \rightarrow n-2 \rightarrow \dots \rightarrow 2 \rightarrow 1$
 $T(n) = O(n)$

CONCLUSION:

Both iterative and recursive methods can be used to calculate the factorial of a number. The iterative method uses a loop and requires less memory, while the recursive method uses repeated function calls. Both have $O(n)$ time complexity, but the iterative method is generally more memory-efficient.